

This homework has 9 questions, for a total of 100 points and 3 extra-credit points.

Unless otherwise stated, each question requires *clear*, *logically correct*, and *sufficient* justification to convince the reader.

We strongly encourage you to typeset your solutions in L^AT_EX.

If you collaborated with someone, you must state their name(s). You must *write your own solution* for all problems and *may not use any other student's write-up or any AI tools*.

0. **Before you start; before you submit.**

If applicable, state the name(s) and username(s) of your collaborator(s).

- (10 pts) 1. **Self assessment.** Carefully read and understand the posted solutions to the previous homework. Identify one part for which your own solution has the most room for improvement (e.g., has unsound reasoning, doesn't show what was required, could be significantly clearer or better organized, etc.). Copy or screenshot this solution, then in a few sentences, explain what was deficient and how it could be fixed.

(Alternatively, if you think one of your solutions is significantly *better* than the posted one, copy it here and explain why you think it is better.)

If you didn't do last week's homework, choose a problem from it that looks challenging to you, and in a few sentences, explain the key ideas behind its solution in your own words.

2. **Counting strings!** Let $C(n)$ denote the number of binary strings of length n that have no consecutive occurrences of 1. For example, $C(3) = 5$; we see this by listing all binary strings of length 3 and checking that only the strings 000, 001, 010, 100, and 101 have no consecutive 1s.

- (2 pts) (a) Compute $C(5)$.

- (4 pts) (b) Derive, with justification, a recurrence relation for $C(n)$. Remember to include appropriate base case(s).

Hint: Consider two cases depending on whether the first character is a 0 or a 1.

- (3 pts) (c) Translate your recurrence into pseudocode that is *top-down recursive*, *without memoization*. You may use [Algorithm 26](#) in the course notes as an example. (Note that this algorithm runs in time exponential in the input value n .)

Finally, show a *lower bound* on the runtime of the algorithm, which establishes that the runtime is exponential in n .

(Note that since the input is n , the input *size* is $\Theta(\log n)$, so this exponential-in- n algorithm runs in *double exponential* time with respect to the input size, i.e. $a^{b^{\log n}}$ for some constants a and b .)

- (3 pts) (d) Translate your recurrence into dynamic-programming pseudocode. ([Algorithm 28](#) in the course notes is a good model.)

In this and subsequent DP questions, credit for pseudocode will be determined based only on how “faithful” (i.e., properly derived) it is to your recurrence—regardless of whether the recurrence itself is correct. (So, pseudocode that isn't faithful to your recurrence will not get full credit—but pseudocode that properly matches an incorrect recurrence *will* get full credit.)

Additionally, for all DP questions, your pseudocode should be written in the “bottom-up” form, rather than top-down memoization.

Analyze the running time of your code, as a function of n . Note that unlike most problems, you are *not* being asked to write the running time in terms of the input size (n is the input value, not the input size.) You may use the fact (without proof) that $C(n)$ has $\Theta(n)$ bits, so adding together two entries of the dynamic programming table takes $O(n)$ time.

3. Seems Familiar?

In this course, pattern matching is a crucial skill. Many problems you will see on homeworks and exams are, at their cores, similar to those you have previously solved in lecture or discussion. Noticing these similarities allows you to use familiar strategies and techniques to solve new problems effectively.

In this problem, you are given 3 dynamic programming problems below. For each of the following parts, you are given another problem that is the same as one of aforementioned problems and can be solved with same recurrence.

For each part, state and briefly justify which problem it is a reformulation of. Explain which aspects of the two problems correspond to each other.

You do not need to provide the recurrence (though you may want to in order to justify your solution).

- **Knapsack Variant:** Given n items where each item i has an associated value v_i and weight w_i , and no two weights are the same, find a subset of items with maximum possible value, whose total weight is at most C .
- **Coin Change:** Given a list of coins $c = [c_1, \dots, c_k]$, where c_i is the value of coin i , and a target amount n , find the number of different collections of coins that have a total value of n .
- **Stair Climbing:** Given a staircase with n stairs, determine the number of distinct ways a person starting at the bottom can reach the top if each time they take a step they can advance either 1 or 2 stairs.

- (4 pts) (a) **Pipe Cutting:** Given a pipe of length L , and a list of pipe lengths with associated prices, find the maximum revenue obtainable by cutting the pipe into pieces, and selling the pieces (discarding any leftover pipe), under the constraint that you may only sell each length of pipe a maximum of one time.
- (4 pts) (b) **Domino Tiling:** Given a $2 \times n$ board, determine the number of ways to completely tile the board using 2×1 dominoes, which can be placed either vertically or horizontally. Every square of the board must be covered by precisely one domino.

4. Minimum Subarray Product

Given an array $A[1, \dots, n]$ of $n \geq 1$ positive real numbers, we are interested in finding the smallest product of any nonempty contiguous subarray of A , i.e.,

$$\min\{A[i]A[i+1] \cdots A[j] : 1 \leq i \leq j \leq n\}.$$

- (8 pts) (a) In this problem you may assume for simplicity that multiplying a pair of real numbers can be done in $O(1)$ time. Give a recurrence relation, including base case(s), that would yield an $O(n)$ -time dynamic-programming algorithm for the smallest product problem. Briefly justify why your recurrence relation is correct. You will analyze the running time of the resulting algorithm in part (b).

Hint: The longest increasing subsequence recurrence might give you some inspiration (but this recurrence should be simpler than that one).

- (6 pts) (b) Give pseudocode implementing an $O(n)$ -time dynamic programming solution to the smallest product problem, and justify the running time. Recall that for simplicity, you may assume that multiplying a pair of real numbers can be done in $O(1)$ time. For all DP problems, remember to refer to the “DP Recipe” from lecture and discussion.

5. Sugar Rush!

You and a friend have won first prize at a Halloween costume party (due to your brilliant Alan Turing and Alonzo Church costumes): a bucket of several pieces of candy, each having some amount of sugar. You and your friend plan to celebrate your win by staying up late and eating all of the candy, but first you need to divide it between yourselves. The problem is that if you and your friend consume different amounts of sugar, the one who eats less sugar will crash first, leaving the other person to stay up alone. You need a way to divide the candy between the two of you so that you both eat the same amount of sugar, and crash at the same time.

In this problem you will give a dynamic-programming algorithm that, given an array $S = [s_1, s_2, \dots, s_n]$ where the i th piece of candy has a non-negative integer s_i grams of sugar, either divides the pieces into two ‘piles’ that have equal amounts of sugar, or (correctly) says that no such piles exist. (Each piece must be in exactly one of the piles.)

- (10 pts) (a) For a given array S , define $P(c, i)$ to be **true** if there exists a pile drawn from the first i pieces of candy having a total of c grams of sugar, and **false** otherwise. Derive, with justification, a recurrence relation for $P(c, i)$. Remember to include appropriate base case(s).

When writing recurrences with Boolean values, you may use operators such as $\wedge$ (and), $\vee$ (or), and $\neg$ (not).

- (3 pts) (b) Briefly state, with justification, the values of c and i to plug into $P(c, i)$, to determine if it is possible to split the candy into two piles having equal amounts of sugar.

- (5 pts) (c) Give pseudocode for a dynamic-programming algorithm that determines whether the candy can be divided into two piles that have the same amount of sugar. Analyze the running time.

Note that this is not an efficient algorithm (in fact, there is no known efficient algorithm for this problem!). However, if each s_i is upper bounded by a constant, then your algorithm should be efficient.

- (4 pts) (d) Not eating the candy is not an option! Given the completed DP table from the previous part, describe in clear English how you would find the *smallest possible difference* in sugar between two piles.

6. Pebble games.

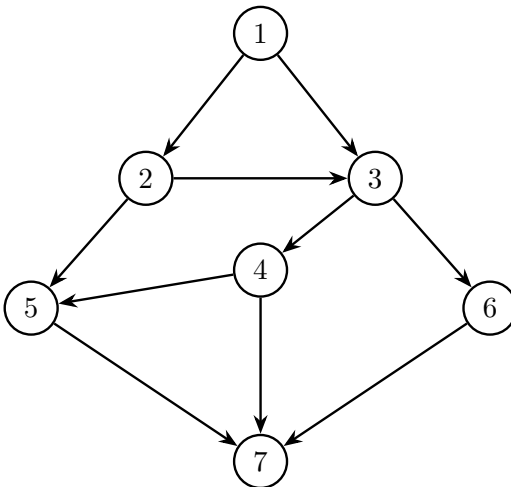
Many two-player strategic games, like the several variants of [Nim](#), can be modeled as follows. There is a directed acyclic graph $G = (V, E)$ presented in “topological” order: the vertices are labeled as $V = \{1, \dots, n\}$, and every edge $(u, v) \in E$ has $u < v$. Moreover, every vertex $i < n$ has at least one outgoing edge.

Two players, called Maize and Blue, take turns in the following game on the graph. They start with a pebble at vertex 1, and Maize plays first. On each turn, the active player (the player

whose turn it is) chooses an edge $(u, v) \in E$ and moves the pebble from its current vertex u to a new vertex v . Unless $v = n$, the other player then becomes the active player with the pebble at v . If a player moves the pebble to vertex n (the final one), then that player wins the game.

It turns out that, for any fixed graph G , one player can be *guaranteed* a win by playing “perfectly”—regardless of what their opponent does. In this problem you will develop a dynamic-programming algorithm that, given the graph G as input, determines *which* player can be guaranteed a win, along with *how* that player can play perfectly.

- (2 pts) (a) In the following graph, there are 7 nodes numbered in topological order. Suppose the pebble is at vertex 1. Determine whether or not the active player (the player who is about to move the pebble) can guarantee a win regardless of what the other player does in future turns. Justify your answer.



- (2 pts) (b) List all vertices in the above graph for which, if the pebble is at that vertex, the active player can guarantee a win. You do not need to justify your answer.
- (8 pts) (c) For a given graph, define $W(i) = \text{true}$ if the active player can be guaranteed a win when the pebble is at vertex i . (Otherwise, $W(i) = \text{false}$.) Derive, with justification, a recurrence relation for $W(i)$. Remember to include appropriate base case(s).

Notation: For representing the Boolean operators $\vee, \wedge$ across many variables, you may use the notation $\bigvee, \bigwedge$. For example $\bigwedge_{i=1}^k x_i = x_1 \wedge x_2 \wedge \cdots \wedge x_k$.

- (5 pts) (d) The algorithm should take as input a directed graph $G = (V, E)$ and determine which of the two players has a guaranteed winning strategy. Design a dynamic programming algorithm that runs in $O(m)$ time, where $m = |E|$. Provide pseudocode for the algorithm and justify its running time.

The input graph $G = (V, E)$ is represented via *adjacency lists*, i.e., for each vertex $i \in V$ there is a list of all its *out-neighbors*: vertices j such that $(i, j) \in E$.

- (5 pts) (e) Describe, in clear and precise English, how to extend your algorithm from the previous part so that it also outputs a “winning strategy” for whichever player has one. This should be represented as an array specifying, for each vertex u , where the player should move the pebble if it is at vertex u on that player’s turn.

7. **Spell Checker** Imagine that you are building a spellchecker for a word processor. When the spellchecker encounters an unknown word, you want it to suggest a word in its dictionary that

the user might have meant (perhaps they made a typo). One way to generate this suggestion is to measure how “close” the typed word A is to a particular word B from the dictionary, and suggest the closest of all dictionary words. There are many ways to measure closeness; in this problem we will consider a measure known as the *edit distance*, denoted $\text{EDIT-DIST}(A, B)$.

In more detail, given strings A and B , consider transforming A into B , via character *insertions* (i), *deletions* (d), and *substitutions* (s). For example, if $A = \text{ALGORITHM}$ and $B = \text{ALTRUISTIC}$, then one way of transforming A into B is via the following operations:

A	L	G	O	R		I		T	H	M
A	L	T		R	U	I	S	T	I	C
		s	d		i		i		s	s

$\text{EDIT-DIST}(A, B)$ is the minimum cost of transforming string A to string B , given the following three positive numbers:

- c_i , the cost to insert a character;
- c_d , the cost to delete a character;
- c_s , the cost to substitute a character.

In this problem you will give and analyze an efficient dynamic programming algorithm that, on input of two strings A, B , computes $\text{EDIT-DIST}(A, B)$ with the costs for insertion, deletion and substitution as c_i, c_d and c_s respectively.

- (8 pts) (a) Give a recurrence relation for edit distance $\text{EDIT-DIST}(A, B)$ and justify its correctness. Remember to include base case(s).

Hint: the recurrence relation for LCS is a good place to look for inspiration.

- (4 pts) (b) Consider the translation of your recurrence into a dynamic programming algorithm. Analyze the running time. While justifying your answer, be sure to include a description of the order in which the DP table should be filled, and where the final answer is located. You are not required to give explicit pseudocode (though you may if you wish).

- (3 EC pts) 8. **Optional extra credit: Wheel deliver.** Jett and Anthony have started a new unicycle-based meal delivery business called “Wheel Deliver” that works as follows. Residents of Ann Arbor submit their requests for a meal each day before noon using the Wheel Deliver App. In the afternoon, Jett and Anthony cook up a delicious meal in their kitchen and deliver the food on their unicycles.

Let n be the number of customers on a given day and $P = (p_1, \dots, p_n)$ be the list of locations of these customers as points in the 2D plane, sorted in the order in which the customers submitted their requests: p_1 is the location of the first person to place an order, and p_n is the location of the last person to place an order. Let p_0 denote the location of the kitchen from which Jett and Anthony depart. Let $d(p_i, p_j)$ denote the distance between p_i and p_j .

Jett and Anthony wish to split up the list of locations P , not necessarily evenly, so that each delivery is made by exactly one of them. In addition, “Wheel Deliver” has a fairness rule that goes like this: If customers at p_i and p_j for $i < j$ are assigned to the same unicycle rider, then the delivery to the customer at p_i must occur before the delivery to the customer at p_j —even if that makes the trip longer. (If p_i and p_j are assigned to different riders, then we don’t care

which delivery occurs first.) Subject to these constraints, the objective is to minimize the total distance travelled by the two unicycle riders.

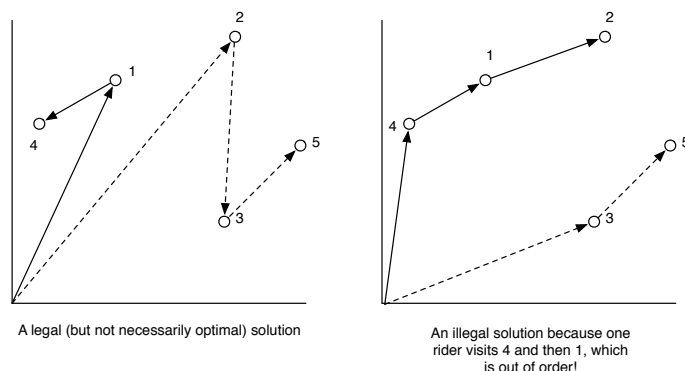


Figure 1: Two examples of routes for the two riders. The origin denotes the starting point p_0 . One rider's route is shown with solid lines and the other with dashed lines. The route on the left is fair because points are visited by increasing indices. The route on the right is not fair because, for example, point p_4 is visited before point p_1 .

Jett and Anthony have hired you to design an efficient algorithm that takes as input an array of points $P = (p_1, p_2, \dots, p_n)$, sorted by the times in which their orders arrived, and outputs the sequences of delivery points for the two riders.

Derive a recurrence relation, including base case(s), that is suitable for an efficient dynamic-programming solution for this problem. Briefly justify its correctness. You do not need to include pseudocode.

Warning: The most natural-seeming recurrences may have subtle errors!